

Федеральное государственное автономное образовательное учреждение высшего
образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Отчёт

По лабораторной работе №2

Сервис-ориентированная архитектура

Вариант: 8965

Выполнили:

студенты 4 курса

Батманов Даниил Евгеньевич 

Группа: Р3407

Принял:

Цопа Евгений Алексеевич

Отчёт принят «__»_____2026 г.

Оценка: _____

г. Санкт-Петербург, 2026

Задание

Внимание! У разных вариантов разный текст задания!

На основе разработанной в рамках лабораторной работы #1 спецификации реализовать два веб-сервиса и использующее их API клиентское приложение.

Требования к реализации и развёртыванию сервисов:

- Первый ("вызываемый") веб-сервис должен быть реализован на фреймворке Spring MVC REST и развёрнут в окружении под управлением сервера приложений Jetty.
- Второй веб-сервис должен быть реализован на фреймворке Spring MVC REST, развёрнут в окружении под управлением сервера приложений Payara и вызывать REST API первого сервиса.
- Для обоих сервисов необходимо реализовать все функции, задокументированные в API, в строгом соответствии со спецификацией!**
- Доступ к обоим сервисам должен быть реализован с по протоколу https с самоподписанным сертификатом сервера. Доступ к сервисам посредством http без шифрования должен быть запрещён.

Требования к клиентскому приложению:

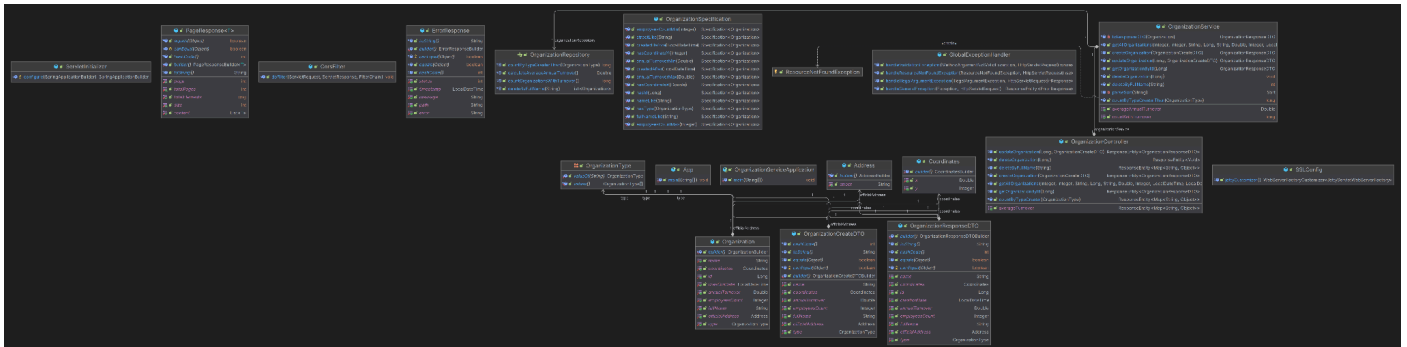
- Клиентское приложение может быть написано на любом веб-фреймворке, который можно запустить на сервере helios.
- Приложение должно обеспечить полный набор возможностей, предоставляемых API обоих сервисов -- включая сортировку, фильтрацию и постраничный вывод элементов коллекции.
- Приложение должно преобразовывать передаваемые сервисами данные в человеко-читаемый вид -- параграф текста, таблицу и т.д.
- Клиентское приложение должно информировать пользователя об ошибках, возникающих на стороне сервисов, в частности, о том, что сервису были отправлены невалидные данные.

Оба веб-сервиса и клиентское приложение должны быть развёрнуты на сервере helios.

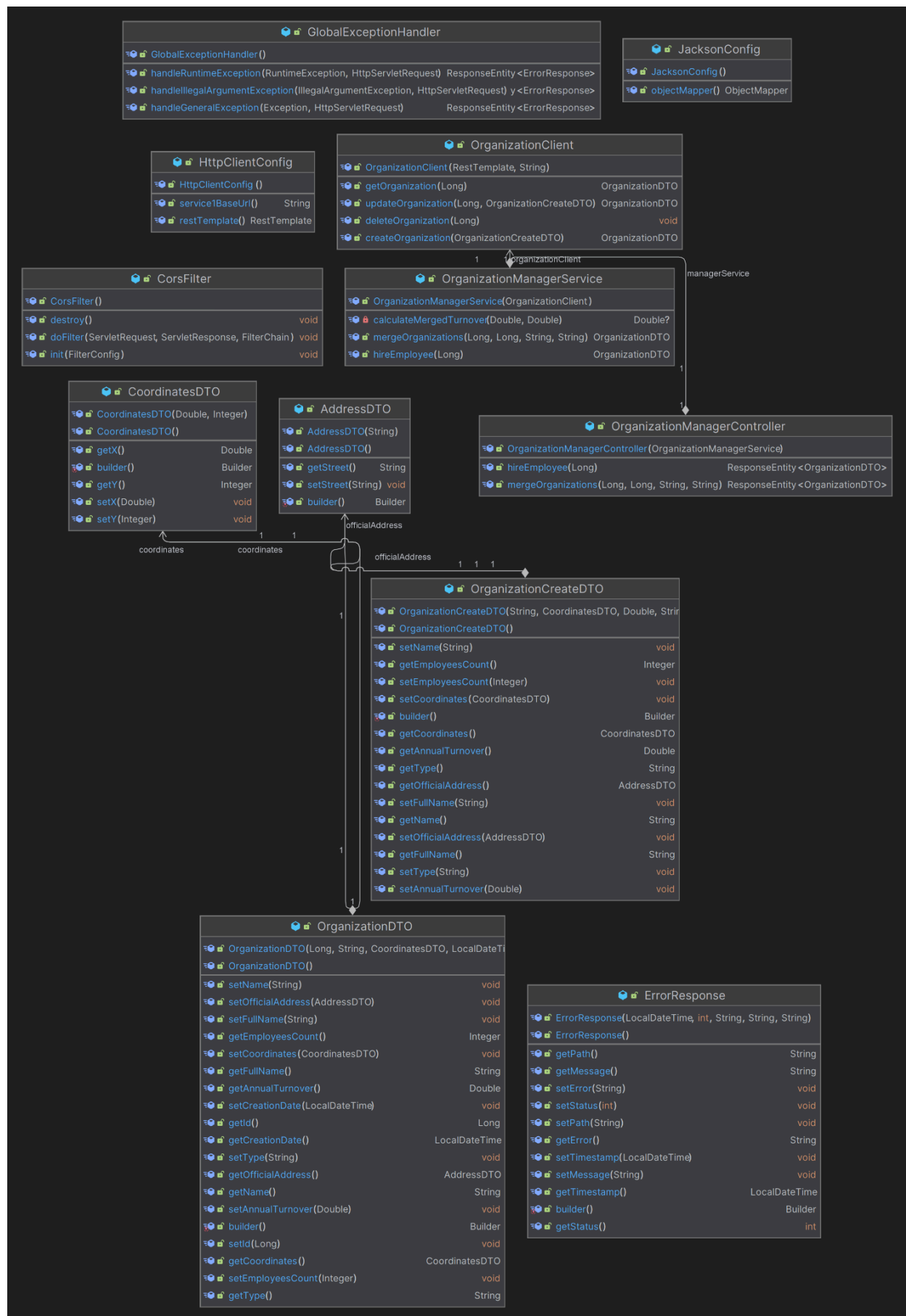
Ход работы

Исходный код:

Сервис 1 - <https://github.com/CodeAxeAttacks/organization-service>



Сервис 2 - <https://github.com/CodeAxeAttacks/organization-manager>



Фронт - https://github.com/CodeAxeAttacks/frontend_soa

Приложение доступно:

<https://se.ifmo.ru/~s367868/soa-lab/>

Создание сертификатов

Первый пользователь:

удалим старые сертификаты

```
rm -f ~/payara-keystore.jks ~/payara-org-service.cer ~/service1-truststore.jks
```

создаём keystore для Service 1 с правильным CN

```
keytool -genkeypair -alias service1 \  
-keyalg RSA -keysize 2048 -validity 365 \  
-keystore ~/service1-keystore.jks \  
-storepass changeit -keypass changeit \  
-dname "CN=helios.cs.ifmo.ru, OU=SOA, O=ITMO, L=SPb, ST=SPb, C=RU"
```

экспортируем сертификат Service 1

```
keytool -exportcert -alias service1 \  
-file ~/service1.cer \  
-keystore ~/service1-keystore.jks \  
-storepass changeit
```

создаём truststore для Service 1 (сюда импортируем сертификат Service 2 позже)

```
keytool -importcert -alias service1-self \  
-file ~/service1.cer \  
-keystore ~/service1-truststore.jks \  
-storepass changeit \  
-noprompt
```

проверка

```
keytool -list -keystore ~/service1-keystore.jks -storepass changeit
```

Второй пользователь:

удалим старые

```
rm -f ~/second-service-keystore.jks ~/second-service.cer ~/payara-keystore.jks  
~/payara-truststore.jks ~/second-service-truststore.jks
```

создаём keystore для Service 2

```
keytool -genkeypair -alias service2 \  
-keyalg RSA -keysize 2048 -validity 365 \  
-keystore ~/service2-keystore.jks \  
-storepass changeit -keypass changeit
```

```
-dname "CN=helios.cs.ifmo.ru, OU=SOA, O=ITMO, L=SPb, ST=SPb, C=RU"
```

```
# экспортируем сертификат Service 2
```

```
keytool -exportcert -alias service2 \  
-file ~/service2.cer \  
-keystore ~/service2-keystore.jks \  
-storepass changeit
```

```
# создаём truststore для Service 2 (сюда импортируем сертификат Service 1)
```

```
keytool -importcert -alias service2-self \  
-file ~/service2.cer \  
-keystore ~/service2-truststore.jks \  
-storepass changeit \  
-noprompt
```

```
# проверка
```

```
keytool -list -keystore ~/service2-keystore.jks -storepass changeit
```

Обмен сертификатами

```
scp s369078@helios.cs.ifmo.ru:~/service2.cer ~/
```

```
# импортируем сертификат Service 2 в truststore Service 1
```

```
keytool -importcert -alias service2 \  
-file ~/service2.cer \  
-keystore ~/service1-truststore.jks \  
-storepass changeit \  
-noprompt
```

```
# проверяем - должны быть ОБА сертификата (service1-self и service2)
```

```
keytool -list -keystore ~/service1-truststore.jks -storepass changeit
```

```
scp s467868@helios.cs.ifmo.ru:~/service1.cer ~/
```

```
# импортируем сертификат Service 1 в truststore Service 2
```

```
keytool -importcert -alias service1 \  
-file ~/service1.cer \  
-keystore ~/service2-truststore.jks \  
-storepass changeit \  
-noprompt
```

```
# проверяем - должны быть ОБА сертификата (service2-self и service1)
```

```
keytool -list -keystore ~/service2-truststore.jks -storepass changeit
```

Танцы с бубном из-за паяры (ssl)

```
# переименовываем алиас service2 в slas
keytool -changealias -alias "service2" -destalias "slas" \
  -keystore ~/service2-keystore.jks \
  -storepass changeit -keypass changeit

# Проверьте, что переименовалось
keytool -list -keystore ~/service2-keystore.jks -storepass changeit
```

Запуск первого сервиса:

```
nohup java -jar ~/org-service-utils/ROOT.jar \
  --spring.config.location=file:/home/studs/s367868/service1-application.yml \
  > ~/service1.log 2>&1 &
```

Создание скрипта запуска второго сервиса

```
# Используем системные свойства для указания SSL конфигурации
java \
  -Djavax.net.ssl.keyStore=/home/studs/s369078/service2-keystore.jks \
  -Djavax.net.ssl.keyStorePassword=changeit \
  -Djavax.net.ssl.trustStore=/home/studs/s369078/service2-truststore.jks \
  -Djavax.net.ssl.trustStorePassword=changeit \
  -Dfish.payara.ssl.override=true \
  -jar /home/studs/s369078/org-manager-utils/payara-micro-6.2025.10.jar \
  --deploy /home/studs/s369078/org-manager-utils/organization-manager.war \
  --contextroot /\
  --autobindhttp \
  --sslPort 18881 \
  --addlibs /home/studs/s369078/org-manager-utils/postgresql-42.6.0.jar \
  --noCluster \
  > /home/studs/s369078/service2.log 2>&1 &
```

```
PID=$!
echo "Service 2 starting with PID: $PID"
echo $PID > /home/studs/s369078/service2.pid
EOF
```

```
chmod +x ~/org-manager-utils/start-service2-fixed.sh
```

Заключение

В ходе выполнения данной лабораторной работы удалось реализовать функционал, заложенный в лабораторной работе №1. Были реализованы 2 сервиса, развернуты на

удалённом сервере. Каждый из сервисов работает под управлением своего сервера приложений jetty/payara.